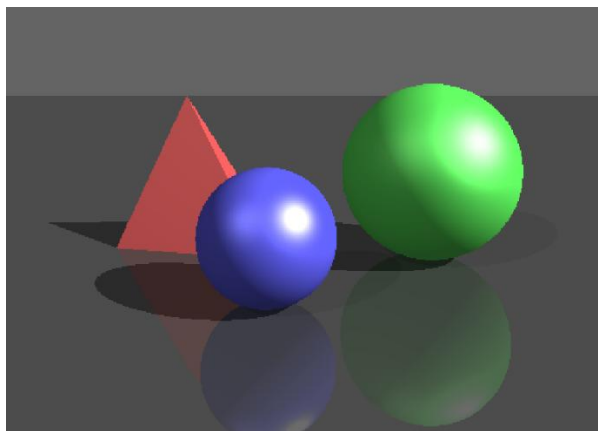


CAP 4730 Raytracer Report – Collin Williams

1. Sample Scene

Below is an image of the final scene that will be rendered (not including the bonus).



It is composed of 2 spheres, 1 tetrahedron, 1 plane, and 2 directional lights. All shape types inherit from a **Shape3D** abstract class, while **Light** is simply a struct.

(All vectors in this report are **bolded**, and scalars are *italicized*)

Shape	Parameters
Sphere	center C , radius <i>r</i>
Plane	point C , normal N
Tetrahedron	points v1 , v2 , v3 , v4
Light	dir , <i>Intensity</i> , <i>kA</i>

All shape types have a *checkIntersection()* method for checking if a ray intersects them, and if so, returning the normal vector at the intersection point. Technically, there are 4 shape classes, since the **Tetrahedron** class is composed of 4 instances of another **Triangle** class. Below are the basic parameters of the scene. Material properties are documented in Section 3.

Parameter	Value(s)
Viewpoint	(1, 0, -12)
LookAt	(-0.082, -0.16, 0.98)
UpVec	(-0.014, 0.99, 0.16)
<i>ViewW</i> , <i>ViewH</i>	6.0
<i>ProjDist</i>	5.0
Sphere1	C = (-2.0, 1.4, -3.0), <i>r</i> = 1.6
Sphere2	C = (1.0, -2.0, -5.0), <i>r</i> = 1.0
Tetrahedron	v1...v4 = (1, -3, -3), (4, -3, -2), (1, -3, 0), (2.5, 0, -2)
Plane	C = (0.0, -3.0, 0.0), N = (0.0, 1.0, 0.0)
Light1	dir = (5.0, -4.0, 3.0), <i>Intensity</i> = 3.0, <i>kA</i> = 0.2
Light2	dir = (-5.0, -5.0, 5.0), <i>Intensity</i> = 0.4, <i>kA</i> = 0.1

2. Camera Behavior

For perspective and orthographic cameras, the implementation is similar enough that a lot of code is shared between the two. For both, the following constants are defined:

Viewpoint a vector representing the camera origin
LookAt the direction the camera is looking
UpVec orthogonal to **LookAt**, controls rotation along the **LookAt** axis
ViewW the width of the viewing window
ViewH the height of the viewing window

Additionally, a *ProjDist* scalar is defined for perspective mode, along with a *DoPerspective* Boolean that allows switching between orthographic and perspective. Given the above definitions, the basis can be calculated:

$$\begin{aligned} \mathbf{e} &= \text{Viewpoint} \\ \mathbf{w} &= -\text{LookAt} \\ \mathbf{v} &= \text{UpVec} \\ \mathbf{u} &= \mathbf{v} \times \mathbf{w} \end{aligned}$$

Now, for every pixel (x, y) on the screen, we need to find the corresponding ray equation for the raytracer. To start, every (x, y) mapped to a vector ranging from $(-0.5, -0.5)$, corresponding to the top-left, and $(0.5, 0.5)$, corresponding to the top-right. These are then transformed into scalar u and v quantities using *ViewW* and *viewH*:

$$\begin{aligned} u &= (x / \text{width} - 0.5) * \text{viewW} \\ v &= (y / \text{height} - 0.5) * \text{viewH} \end{aligned}$$

The mapping is done in this way so the center of the screen is mapped to $(0, 0)$, and the width / height between the endpoints matches *viewW* and *viewH*. Now, when calculating the **S** and **D** for the ray equation $\mathbf{R}(t) = \mathbf{S} + t\mathbf{D}$, the perspective and orthogonal cameras have different implementations. For the perspective camera, the **S** vector is always **e**, and the **D** vector varies based on (u, v) . For the orthogonal camera, the **S** vector varies based on (u, v) , and the **D** vector is always **-w**.

Perspective:

$$\begin{aligned} \mathbf{S} &= \mathbf{e} \\ \mathbf{D} &= -\mathbf{w} * \text{ProjDist} + \mathbf{u} * u + \mathbf{v} * v \end{aligned}$$

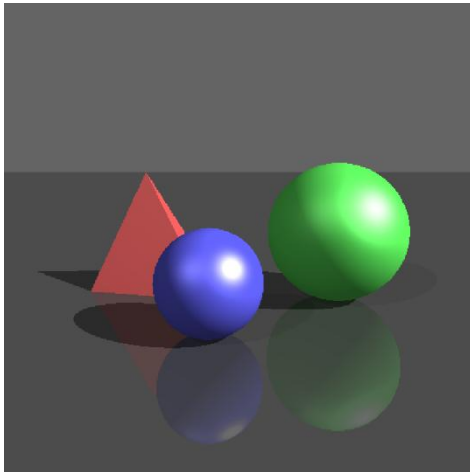
Orthogonal:

$$\begin{aligned} \mathbf{S} &= \mathbf{e} + \mathbf{u} * u + \mathbf{v} * v \\ \mathbf{D} &= -\mathbf{w} \end{aligned}$$

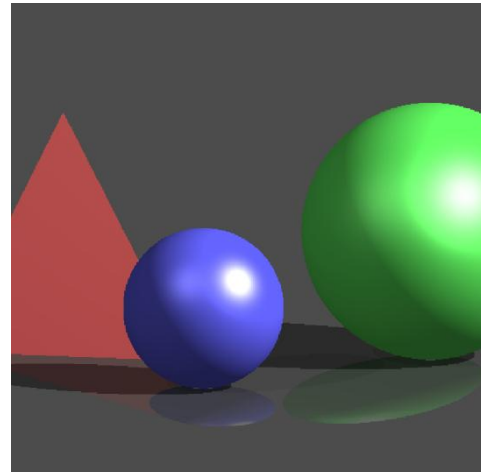
Also, the **D** vector is normalized to simplify calculations later on. While the program is running, the P key can be used to toggle between perspective and orthographic. Additionally, the WASD keys move the camera parallel to the ground, SPACE moves it up, and SHIFT moves it down. E and R rotate the camera around the y-axis.

The images below show a comparison of perspective and orthographic cameras:

Perspective:



Orthogonal:



3. Lighting

As indicated in Section 1, there are two directional lights in the sample scene. Each object in the scene contains its own instance of a **Material** struct, which controls information about color and light behavior. Material allows modifying the following properties:

Color	The base color of the object
<i>kD</i>	Diffuse coefficient
<i>kS</i>	Specular coefficient
<i>N</i>	Blinn-Phong shading exponent
<i>Glaze</i>	Controls the proportion of color that is sampled from reflected light; defaults to negative, reflected light is only probed if between 0 and 1
<i>Translucent</i>	Boolean value allowing translucent material (bonus)

The scene's materials are composed as follows:

Object	Color	<i>kD</i>	<i>kS</i>	<i>N</i>	<i>Glaze</i>
Red Sphere	(255, 0, 0)	0.3	0.1	10.0	-1.0
Green Sphere	(0, 255, 0)	0.3	0.2	20.0	-1.0
Blue Sphere	(0, 0, 255)	0.3	0.3	30.0	-1.0
Floor Plane	(80, 80, 80)	0.3	0.0	10.0	0.2

3.1 Ambient Lighting & Shadows

Each light in the scene has an ambient coefficient kA associated with it. When sampling the lighting at a point, this coefficient is always added, regardless of any other properties of the scene. Thus, for any given pixel, the minimum intensity is the sum of the ambient coefficient of all the lights in the scene. This might not model light fully realistically, especially for objects with many lights, but it works well for our example.

The way I modeled lighting, I kept track of 2 different variables: *Intensity* and *SpecularIntensity*. Each intensity affects color in a different way: *Intensity* models color ranging from black to the source color, and *SpecularIntensity* starts at the source color and makes the object whiter. Ambient lighting only affects *Intensity*, resulting in a final color that's a mix of the object color and black.

$$Intensity = kA$$

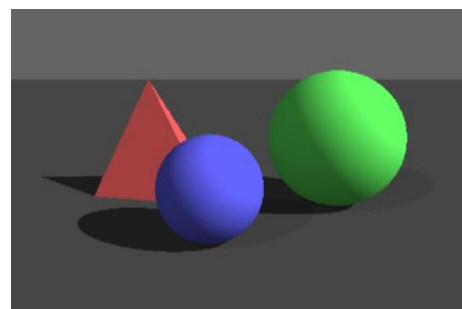


Now, every point in the scene receives a dim amount of lighting. Before we calculate the diffuse and specular lighting, we should test if rays from the light are actually able to reach the given point. If not, the diffuse and specular lighting is ignored, and a shadow is formed. This is done using the same *checkIntersection()* method used when originally sampling for the intersection point.

3.2 Diffuse Lighting

For points that light rays are able to reach, we calculate the diffuse lighting using the dot product of the object normal and the light direction $\mathbf{vL}$. Points with a normal that closely aligns with the light rays are bright, and points with normals orthogonal to the light rays are dark. To ignore points facing away from the light, a *max()* function is used to ignore negative dot products. As a reminder, the diffuse coefficient kD is a parameter that each object contains in its material.

$$Intensity = kA + kD * LightIntensity * \max(0.0, \mathbf{Normal} \cdot \mathbf{vL})$$



3.3 Specular Lighting

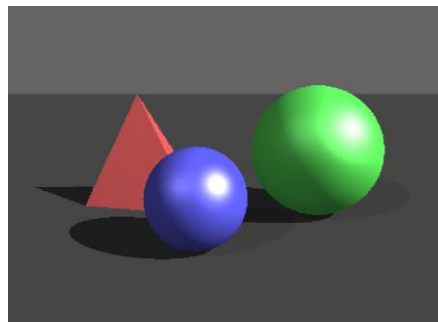
This project using the Blinn-Phong approximation for specular lighting. This means that the bisector $\mathbf{vH}$ is calculated using:

$$\mathbf{vH} = \text{normalized}(\mathbf{vL} + \mathbf{vE})$$

Then, specular intensity is calculated in a similar way to diffuse intensity. The specular intensity is added to both *Intensity* and *SpecularIntensity*. As with kD , the kS and N coefficients are both stored in the object's material.

$$\begin{aligned} \text{SpecIntensity} &= kS * \text{LightIntensity} * \max(0.0, \mathbf{Normal} \cdot \mathbf{vH})^N \\ \text{Intensity} &= \text{AmbientIntensity} + \text{DiffuseIntensity} + \text{SpecIntensity} \end{aligned}$$

As mentioned before, the *SpecularIntensity* adds white on top of the already-existing object color:



3.4 Glazed Surfaces

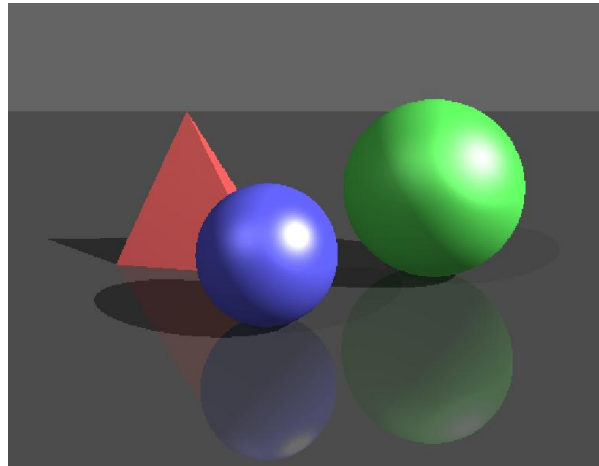
The appearance of glazed surfaces is controlled by a float in the object's material. For the floor plane, a *glaze* of 0.2 was chosen, meaning 20% of the object's color output is sampled from reflected light rays. To model the physics of light reflection, we start with the object's **normal** and the $\mathbf{vE}$ vector, which represents the direction the ray originated from. Then, the reflected vector $\mathbf{vR}$ can be found using:

$$\mathbf{vR} = 2 * \mathbf{normal} * (\mathbf{normal} \cdot \mathbf{vE}) - \mathbf{vE}$$

Now, we have a light ray with an origin point (the intersection position), and a direction ($\mathbf{vR}$), so we can repeat these same steps to sample the color from this new ray's perspective. Conveniently, I abstracted all the raytracing logic into a function called *getRayColor()*, which is able to call itself recursively. The function keeps track of the recursion level, which prevents an infinite loop. When the color from the reflected ray is found, the final color is simply a mix of the surface color and reflected color:

$$\text{Color} = \text{SurfColor} * (1 - \text{glaze}) + \text{ReflectedColor} * \text{glaze}$$

This is the final result with all shading methods implemented:



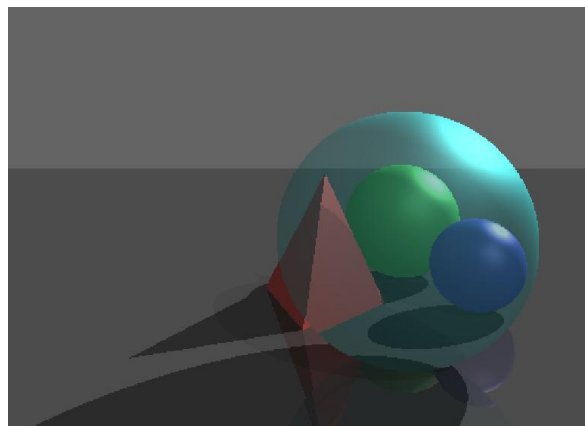
4. Video Rendering

Included with the project are 2 videos. The first is a demo of the sample scene with camera movement. The second demonstrates the light refraction behavior I implemented as a bonus.

Both videos were rendered using parametric equations defining camera movement. For each frame, the current image was exported using the SFML library. SFML is not needed to link the provided project, but can be linked by defining `#include sfml` in `RayTracer.h` and using `make sfml_link` in the terminal. Finally, all the exported images were stitched together using FFmpeg in the terminal.

5. Translucent Materials and Light Refraction (Bonus)

For a bonus feature, I implemented a sphere with a translucent material. This material will also diffract light waves that pass through it. The sphere is centered at $(5.0, -1.0, -4.0)$, has radius 2, is blue, and has the flag *Translucent* set to *true* in its material. A visual is shown below, but I feel like the video demonstrates the refraction better.



The implementation isn't super accurate to how refraction works in physics, but it was cheap to implement and gave a convincing result. It works similarly to glazed surfaces, where a secondary ray is cast and the probed color is mixed with the source color of the object. The only difference is that the ray is cast in the same direction it arrived at the sphere from. However, instead of casting the ray from the intersection point, the **LookAt** vector is multiplied by 0.6 and added to the position. This is what gives the effect of shifting light without giving a result that looks inaccurate.

This probed color is mixed with the sphere's color based on its *Alpha* value. The *Alpha* ranges from 0.2 to 0.8, depending on light. I found it looked best when light intensity reduced the transparency, giving it the appearance of glare:

$$\begin{aligned} \textit{Alpha} &= 0.2 + \textit{Intensity} * 0.6 \\ \textbf{Color} &= \textbf{SurfColor} * \textit{alpha} + \textbf{RayColor} * (1.0 - \textit{alpha}) \end{aligned}$$